

OFFICIAL RULES



Tankie

Complete game rules and protocol reference

1 · Overview

Tankie is a **turn-based, 2-player AI tank battle**. Two bots compete on a procedurally generated maze. Each bot runs as a separate client that connects to the Godot game server over WebSocket. Players alternate turns; on your turn you submit a list of actions. The last tank standing wins.

2 · Turn Structure

RULE 2.1

Players take turns in strict alternation.

The server decides who goes first (`onTurn` in the `game_started` event). After each turn a `turn_changed` event announces the next player.

RULE 2.2

On your turn you send exactly one action list.

Send a JSON message with your `tankId` and an `actions` array. The server processes it, broadcasts events for what happened, then switches to the other player.

RULE 2.3 — ACTION BUDGET

Each turn has a budget of at most 2 move/shoot tokens.

Move costs **1 token**. Shoot costs **1 token**. Rotate costs **0 tokens** (always free, any number).

Valid combinations: 2 moves · 1 move + 1 shoot · 1 shoot · 1 move · rotates only.

Over-budget submissions are silently rejected. If your action list exceeds the budget, the server discards the entire message and your turn is skipped — you receive no error.

3 · Actions

Move

Moves the tank one grid cell in the given direction. The tank must not be blocked by a wall.

```
{ "type": "move", "direction": "up" | "down" | "left" | "right" }
```

If a wall blocks the path, the move is ignored (the token is still consumed).

Rotate

Rotates the turret by the specified number of degrees. Any integer value is accepted. Rotation is free — it does not consume a budget token.

```
{ "type": "rotate", "degrees": 90 }
```

You may include multiple rotate actions in a single turn at no cost.

Shoot

Fires a bullet from the tank's current position in the turret's facing direction.

```
{ "type": "shoot" }
```

Costs 1 token. Only one shoot action per turn is allowed.

4 · Bullet Physics

RULE 4.1 — BOUNCE LIMIT

A bullet bounces off walls up to 2 times.

After the second bounce the bullet despawns regardless of its remaining travel distance.

RULE 4.2 — ELIMINATION

A bullet that hits a tank eliminates it immediately.

There is no health system — one hit is one kill.

RULE 4.3 — SELF-DAMAGE

A bullet can bounce back and hit the shooter.

Plan accordingly when shooting near walls.

5 · Win Conditions

Situation	Result
One tank is hit by a bullet	The other tank wins
Both tanks are eliminated by the same bullet (via bounce)	Draw — <code>winner</code> field is an empty string
A tank shoots itself via bounces	The opponent wins

The server broadcasts a `game_over` event with the winning `tankId` (or empty string for a draw).

6 · Coordinate System

The map is a rectangular grid of cells. The origin `(0,0)` is the top-left corner. X increases to the right; Y increases downward.

```
(0,0) — x →
|
y
|
▼
```

Grid position

Integer cell index used in the `map` event and state tracking.

```
grid_x = floor(world_x / gridSize)
```

World position

Pixel coordinate reported in `moved` events. Points to the center of the cell.

```
world_x = grid_x * gridSize + gridSize / 2
```

Default `gridSize` : **128 pixels** per cell.

7 · Walls

A wall occupies the **edge between two cells**, not the cell itself.

Orientation	Position (x, y)	Separates
HORIZONTAL	(x, y)	Cell (x,y) from cell (x, y+1) — wall on the <i>bottom</i> edge of (x,y)
VERTICAL	(x, y)	Cell (x,y) from cell (x+1, y) — wall on the <i>right</i> edge of (x,y)

Boundary walls (map edges) use out-of-range coordinates, e.g. `y = -1` for the top border.

Checking for a wall in your strategy:

```
state.has_wall(x, y, "HORIZONTAL") # Python
state.HasWall(x, y, "HORIZONTAL") // C#
state.has_wall(x, y, "HORIZONTAL") // C++
```

8 · Server Events Reference

Event	Key fields	When sent
<code>player_joined</code>	<code>tankId</code>	A client connected and joined the lobby
<code>map</code>	<code>gridWidth</code> , <code>gridHeight</code> , <code>gridSize</code> , <code>walls[]</code> , <code>players[]</code>	Once, just before <code>game_started</code>
<code>game_started</code>	<code>onTurn</code>	Countdown ended, first turn begins
<code>turn_changed</code>	<code>nextTurn</code>	After every completed (or skipped) turn
<code>moved</code>	<code>tankId</code> , <code>direction</code> , <code>pos_x</code> , <code>pos_y</code>	A tank successfully moved (world coords)
<code>rotated</code>	<code>tankId</code> , <code>degrees</code>	A tank's turret rotated
<code>shot</code>	<code>tankId</code>	A tank fired a bullet
<code>game_over</code>	<code>winner</code> (empty = draw)	The game ended

9 · Disqualification

OVER-BUDGET

Submitting more actions than the budget allows skips your turn.

No error is returned. The server simply advances to the next player.

TURN TIMEOUT**If a player does not respond within the time limit, your turn is skipped.**

A `turn_timeout` event is fired, then `turn_changed` advances the game.

REPEATED DISQUALIFICATION**If a player is disqualified too many times, they are eliminated.**

The server fires a `turn_disqualified` event and marks the tank as dead.